

Aravind Pradeep

AI Engineer | Agentic Systems & RAG Pipelines | Full-Stack GenAI

Cottbus, Germany | +49 176 67314504 | aravindpradeep001@gmail.com

linkedin.com/in/aravind-pradeepmadathinal | github.com/axon011 | aravindpradee.me

English (C1) | German (B1)

PROFESSIONAL SUMMARY

AI Engineer with two years building the backend and model-evaluation layer of an industrial-IoT AI platform at Perinet — Python and Go services over live MQTT streams behind FastAPI, Docker, and Kubernetes, plus the benchmarking suite that measured retrieval speed against generation quality across model variants. On my own projects I build the RAG, agent, and LLMops side: hybrid retrieval (BM25 + dense with reciprocal rank fusion) reaching 0.94 hit@5, RAGAs and MLflow evaluation, LangGraph agent pipelines, and call-level observability — shipped publicly, including a live multi-agent app at german-tutor.vercel.app. Nearly three years of professional software experience in total, including earlier enterprise work at Cognizant. Finishing my M.Sc. in AI at BTU Cottbus, available for full-time work on thesis completion.

PROFESSIONAL EXPERIENCE

AI Engineer (Working Student)

Perinet GmbH | Cottbus, Germany | Jul 2024 – May 2026

- Developed Python and Go backend services connecting LLM workflows to live MQTT sensor streams, exposed as versioned FastAPI REST APIs with async endpoints; containerized with Docker/Kubernetes and automated CI/CD via GitHub Actions for zero-touch deployments.
- Led the containerization workstream for the AI Container umbrella project — PeriChat (Go-based chatbot with MapReduce retrieval), MessageSense (streaming MQTT message anomaly detection), and PeriXplore (sensor data exploration); profiling, QA, and deployment integration in collaboration with the engine and ML teams.
- Owned the model-benchmarking workstream; evaluated retrieval speed, generation quality, and trade-offs across model variants for the AI container platform — informed downstream chatbot and corpus design.

Software Engineer Trainee

Cognizant Technology Solutions | India | Oct 2021 – May 2022

- Developed and maintained mainframe applications (COBOL, JCL, DB2) in agile sprints; built structured debugging practices and production deployment discipline across enterprise banking systems.

PROJECTS

Deutsch-Tutor: Multi-Agent Language Learning App | Next.js, TypeScript, React, SSE Streaming, Zod, Claude/Gemini APIs | Live: german-tutor.vercel.app

- Built and deployed (Vercel) a full-stack multi-agent German tutor in Next.js (App Router) and TypeScript: a Conversation agent streams replies over SSE while a parallel Corrector agent returns Zod-validated grammar-error JSON rendered as inline annotations — corrections never block time-to-first-token.
- Cut warm-turn time-to-first-token from about 18s to under 1s by replacing spawn-per-turn LLM calls with a persistent streaming session behind a swappable provider interface; a rule-based recommender (frequency × recency decay over logged errors) turns the learner's own mistakes into personalized drills across a 32-lesson CEFR curriculum and 40-rule grammar reference.

Wind-Farm Installation Planner | Python, NumPy, pandas, Pydantic, FastAPI, Simulation & Optimization | github.com/axon011/windfarm-planner

- Built a weather-aware construction scheduler for a 12-turbine wind farm — a resource-constrained project scheduling problem where crane lifts are uninterruptible, wind-limited, and confined to working hours — searching 48 candidate campaigns and pricing each against 20 years of hourly wind as a P50/P90 distribution over 19 weather-years instead of a single estimate; the recommended plan cut cost from EUR 5.59M to EUR 3.56M and duration from 189 days to 64.
- Kept all scheduling deterministic and used the LLM only at the human edges: plain-English operating rules are parsed into typed constraint specs the user approves before they compile into checks that the scheduler and an independent auditor both enforce. Verified three ways — per-schedule constraint audit, a clean-room cost recompute matching the engine to the euro, and 15 automated tests.

Multi-Agent Research & Report Pipeline | LangGraph, CrewAI, FastAPI, Docker, GitHub Actions

- Architected 3-agent pipeline (Planner, Researcher, Writer) using LangGraph state machines with strict role

boundaries and Pydantic structured outputs, producing 2,000+ word research reports with verifiable source citations.

- Deployed via async FastAPI with Docker containerization and GitHub Actions CI/CD; automated end-to-end from code push to deployment with zero manual intervention.

RAG Evaluation System | FastAPI, Qdrant, LangChain, RAGAs, MLflow, BM25

- Engineered hybrid retrieval pipeline (dense embeddings + BM25 + Reciprocal Rank Fusion) over a 50-topic corpus using Qdrant, reaching 0.94 hit@5 and 0.96 citation presence on a 50-question evaluation set, with async embedding pre-computation and semantic caching.
- Built automated evaluation harness benchmarking faithfulness, relevance, and context recall via RAGAs; tracked 50+ prompt experiments in MLflow with automated regression alerts on retrieval quality degradation.

GraphRAG Studio | Python, networkx, Next.js, TypeScript, React, FastAPI, Docker

- Built a GraphRAG pipeline that constructs a typed, deduplicated knowledge graph via LLM entity/relation extraction (networkx), with k-hop subgraph retrieval producing grounded, source-cited answers to multi-document and relationship-spanning questions that flat RAG misses.
- Shipped it as a full-stack application: a Next.js (App Router) and TypeScript front end with an interactive force-graph visualization over a FastAPI backend — upload documents, watch the graph build, then chat over it.

LLM Fine-Tuning: Structured JD Extractor | QLoRA, Qwen2-0.5B, PyTorch, PEFT, 4-bit Quantization

- Fine-tuned Qwen2-0.5B with QLoRA (4-bit NF4, LoRA rank 16 / alpha 32 on attention projections) to extract structured JSON from job descriptions; reached 100% JSON validity and 70%+ entity-field accuracy on a held-out test set, trained in under 4 minutes on a 4GB GPU.
- Trained 2.16M adapter parameters (0.44% of 496M total) and evaluated per-field accuracy across entity and list fields, exposing where list-field F1 needed more training data.

LLMOps Observability Dashboard | FastAPI, React, TypeScript, PostgreSQL, Docker, GitHub Actions

- Built a full-stack LLM observability dashboard that traces every API call to PostgreSQL with latency, token counts, and per-model cost (GPT-4o, Claude, Gemini, DeepSeek) — self-hosted, no external tracing service — with a React + Recharts session explorer for prompt previews and per-session cost breakdowns.

SKILLS

AI & Agents: LangGraph, LangChain, CrewAI, RAG, Prompt Engineering, Structured Outputs (Pydantic), Tool Use, Function Calling, MCP

LLMOps: Langfuse (Tracing, Evals), RAGAs (Faithfulness, Relevance, Recall), MLflow, LLM-as-Judge, A/B Prompt Evaluation, Guardrails

ML & NLP: Python, PyTorch, Hugging Face Transformers, scikit-learn, OpenAI/Anthropic APIs, Open-Source LLMs (Qwen, DeepSeek, Llama), Fine-Tuning, Embeddings, NER, CV

Backend & Infra: Go, FastAPI, REST/gRPC, Docker, Kubernetes, GitHub Actions, GitLab CI/CD, Azure, AWS (S3, Bedrock, EC2), Qdrant, ChromaDB, OpenSearch, Elasticsearch, PostgreSQL, React, TypeScript

Languages: English (C1 Fluent), German (B1 Actively Improving), Malayalam (Native)

EDUCATION

M.Sc. Artificial Intelligence (Research Profile)

Brandenburg University of Technology | Cottbus, Germany | Oct 2022 – present (thesis phase)

- Thesis: Content-Aware ViT Optimization on Edge Devices. Focus: ML, Computer Vision, Explainable ML, Data Mining.

B.Sc. Computer Application

BVM Holy Cross College | Kottayam, India | Jul 2018 – Mar 2021